TCGA-UT Imbalanced

Julius, Laure, Augustin

February 2026

1 Motivation/Introduction

- in natural histopathology datasets, severe imbalances often occur
- rare diseases often have orders of magnitude fewer samples than common diseases
- consequently, foundation model features of these rare classes tend to be worse-separated, and down-stream classification performs worse on these classes
- in addition, models tend to be overconfident on these examples, something known as bad calibration
- another issue is that models often tend to be confidently wrong, i.e., they assign high probability scores to wrong classes
- this is known as bad calibration
- on the flip side, one talks of good calibration if probability scores assigned to classes by models reflect the model's accuracies on these classes
- methods exist aiming to
  a) increase performance on rare classes in imbalanced settings
  b) improve calibration such that worse performance on rare classes is reflected in less confident scores on these classes
- we introduce a controllable imbalanced histopathology dataset that allows the user to control the degree of imbalance with a single parameter
- we do this by mapping the class distribution onto a power law, which is explained in detail in Section 2.2.3
- we aim to use this to investigate:
  a) robustness of foundation models to imbalancedness, i.e., how well-separated are features of rare classes

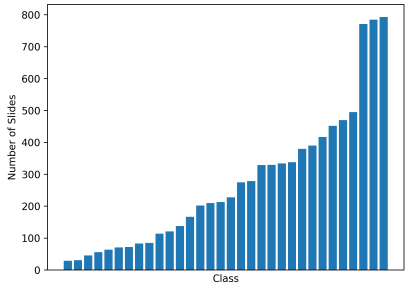


Figure 1: Number of slides per class in TCGA-UT

b) how does imbalance affect the performance of downstream classification heads

c) how well do different strategies mitigate imbalances

d) how well do different strategies help calibration

2 Dataset Design

2.1 TCGA-UT

- our dataset is based on the TCGA-UT data set [1]
- TCGA-UT contains over 1.6 million patches of size 256x256 pixels
- derived from TCGA
- it encompasses 32 different cancer types, and per cancer type patches from a variable number of slides
- a variable number of regions have been extracted from the slide's tumor areas, each consisting of 10 distinct patches
- notably, TCGA-UT is not itself balanced, as can be seen from Figure 1

2.2 Sampling

2.2.1 Balanced Sampling

- because TCGA-UT is itself not balanced, we first sample a balanced version of TCGA-UT, which we will refer to as our TCGA-UT base set
- to do this, we first decide on a number N_b of slides per class in the balanced version of TCGA-UT

- For each class, we randomly select N_b distinct slides (without replacement). Classes for which less than N_b slides are available are excluded.
- we then select the first P_b patches ordered by group and patch number within that group
- to make sure only entire groups of patches are selected, P_b should be divisible by 10
- in our particular setting, we set $N_\mathrm{b} = 100$ and $P_\mathrm{b} = 30$
- the resulting TCGA-UT-balanced consists of 23 different classes, with 3000 patches per class, i.e., 69000 patches in total

2.2.2 Balanced Validation Dataset

- before we sample the imbalanced version of TCGA-UT, we sample a balanced validation set from TCGA-UT-balanced
- for this, we set a number of slides per class in the validation set and randomly select these from each class present in TCGA-UT-balanced
- the selected slides are then removed from the pool of slides used in the subsequent imbalanced sampling step
- notably, this step is seeded with the same seed as in the subsequent imbalanced sampling
- thus, the validation dataset will always be the same, given the same seed, independent of all other parameters used in the imbalanced sampling
- this ensures comparability across parameters in the imbalanced setting

2.2.3 Imbalanced Sampling

- to create an imbalanced version from TCGA-UT-balanced, we subsample according to a power-law distribution
- to do this, we first assign an order to the classes, which determines which classes get sampled most and which get sampled the fewest
- specifically, each class receives a unique index from $\{1, \ldots, C\}$, where a class gets sampled more often, the lower its index is
- the probability of sampling a slide from the class with index i can be seen in Eq. 1

$$p_i^\lambda = \frac{i^{-\lambda}}{\sum_{j=1}^{C} j^{-\lambda}} \tag{1}$$

- λ is a parameter that controls the degree of imbalance

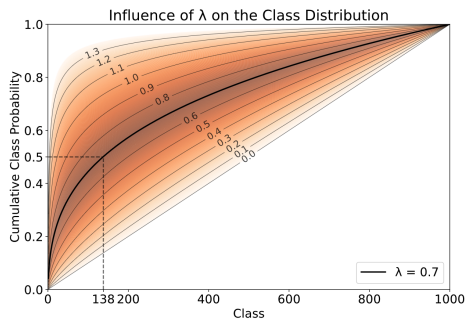


Figure 2: Powerlaw at different parameters. (Taken for visualization purposes from the Master's thesis "Open-Set Recognition on Imbalanced Datasets" by Leo Pinetzki)

- as λ increases, sampling a class with a low index becomes increasingly more likely, while higher class indices get sampled less frequently
- the power-law distribution at different parameters can be seen in Figure 2
- notably, setting $\lambda = 0$ results in a mostly balanced set that might still be slightly imbalanced, since it is also has to satisfy the constraint of a total data set size, which might not be divisible by the number of classes in our base set

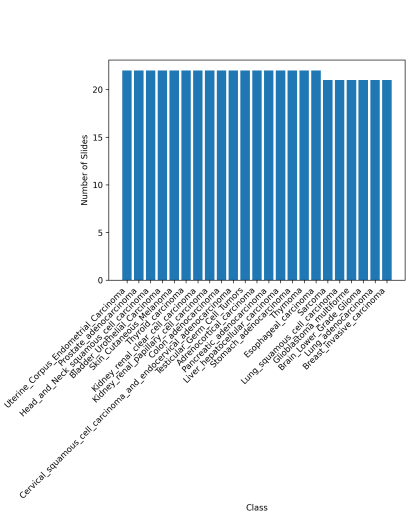
- to make the imbalanced datasets sampled at different parameters comparable, we fix their size to $N = 500$ slides
- to obtain the number of samples for a particular class under a particular parameter, we multiply $N_i^\lambda = N \cdot p_i^\lambda$
- since this might be non-integer, we floor this number and obtain the fractional remainders of each class $f_i^\lambda = N_i^\lambda - \lfloor N_i^\lambda \rfloor$
- we then add an additional sample to each class in descending order of their fractional remainders until the total number of samples equals N
- optionally, one can set the number of groups selected from each slide, and the number of patches selected from each group
- another problem might arise: the number of required samples per class can exceed the number of available samples per class, i.e., $N_i^\lambda > N_b$
- in this case, the distribution $p_i^\lambda, i = 1, \ldots, C$ is not feasible given the constraints that $N = n$ and $N_b = n_b$ impose
- to be able to sample at λ outside of the "feasible region" of parameters, we map the resulting distribution onto the closest feasible distribution

- we do this iteratively in each iteration,
  - we first set $N_i^\lambda = \max(N \cdot p_i^\lambda, N_b)$ for $i = 1, \ldots, C$
  - let $\mathcal{C} = \{i : i = 1, \ldots, C\}$ and let $\mathcal{O} = \{i : N_i^\lambda = N_b, i \in \mathcal{C}\}$
  - then we update $N^{\text{new}} \leftarrow N^{\text{old}} - |\mathcal{O}| \cdot N_b$ and recalculate $p_i^{\lambda,\text{new}}$ with N^{new} samples and over the classes $\mathcal{C}/\mathcal{O}$
  - we repeat this until no classes overflow
- the resulting distribution is closest to the desired power-law distribution in KL-divergence (proof missing)
- Figure 3 shows the number of samples per class at a few different parameters

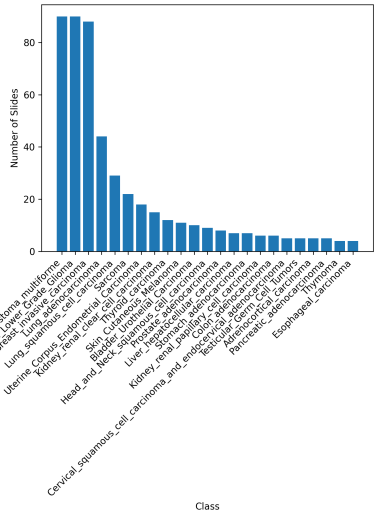
3 Evaluation

3.1 Two Axes: Class difficulty versus Frequency

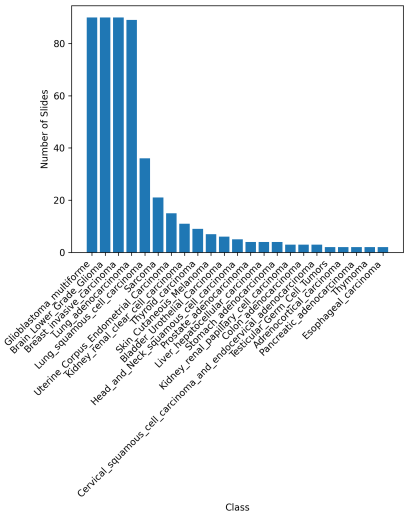
- any metric evaluating the performance of a model on a specific class will depend on two major factors:
  - Class frequency: the number of available samples to learn from
    - this is controllable by the parameter λ in our sampling approach
  - Class difficulty: The challenge of separating a class from other classes in (foundation model) feature space
    - e.g., the features might be strongly overlapping with the features of other classes
- any evaluation of the performance of a trained model has to take these two factors into account
- the evaluation in different settings also depends on randomness in the following ways:
  1. slides get randomly sampled in the creation of TCGA-UT-balanced
  2. slides get randomly sampled in the creation of TCGA-UT-imbalanced
  3. the initialization of the model weights and the shuffling of the batches by the training data loader are also random
- this implies that our evaluations have to be run multiple times, with different randomness seeds, in order to obtain reliable estimates of performance metrics



(a) $\lambda = 0.0$



(b) $\lambda = 1.0$



(c) $\lambda = 1.3$

Figure 3: The distribution over the classes at a few different parameters.

3.1.1 Evaluating Inherent Class Difficulty

- to understand the inherent difficulty of the class features, we train simple classifiers/estimators on top of the frozen foundation model features in the completely balanced setting ($\lambda = 0$)
- by training simple classifiers (e.g., KNN, NCC, or a simple one-layer MLP) on frozen foundation model features we can investigate the linear separability of the features
- thus, the recall values per class of the simple classifier can give an indication of class difficulty

3.1.2 Evaluating Difficulty of Learning Underrepresented Classes

- to evaluate how a trained model performs at different degrees of imbalance, we mainly investigate recall values of classes under different parameters, as well as confusion matrices

3.1.3 Interaction of Class Difficulty and Frequency

- the two axes outlined above jointly influence the performance of a model on each class, and it is often difficult to isolate the role of each factor in a specific prediction performance on a class
- to be able to control a class' difficulty in conjunction with the number of samples it gets, we derive three settings (orders in which the classes are sampled from largest to smallest) from the KNN-based results in the $\lambda = 0$ setting:
  1. "Original Class Size Order": the classes are ordered by the number of slides of their class in the original TCGA-UT. This aims to represent a realistic class ordering.
  2. "Easy to Difficult": the classes are ordered by the recalls achieved on them by the KNN-based classifier, from highest to lowest. This means that easy classes will be oversampled and difficult classes will be undersampled.
  3. "Difficult to Easy": the classes are ordered by the recalls achieved on them by the KNN-based classifier, from lowest to highest. This means that difficult classes will be oversampled and easy classes will be undersampled.
- in the second setting, classes that are more easily classified by a linear classifier are also overrepresented in the imbalanced data sets, while more difficult classes will be underrepresented
- in the third setting, the class order is flipped from the Easy to Difficult setting

3.2 Mitigation Strategies

- various strategies have been proposed to help models learn and perform better in imbalanced settings
- these strategies include:
  1. Batch Balancing
  2. Loss Reweighting based on Class Frequency
  3. Focal Loss (with and without class frequency reweighting)
  4. OkO

3.2.1 Batch Balancing

- batch balancing compensates for imbalances in the data set by sampling each batch from the training data set uniformly over the classes
- in order to ensure the same batch size as in the other settings, we have to sample with replacement, since small classes have too few samples to sample uniformly without replacement
- consequently, the concept of an epoch does not apply anymore
- however, to ensure comparability with the other methods, we ensure that each trained model has performed the same number of update steps

3.2.2 Weighted Cross Entropy

- this method multiplies the (cross entropy) loss $\mathcal{L}(\mathbf{x}, y) = -\log p_y(\mathbf{x})$ of each sample by a weight proportional to the inverse of its class frequency in the training data set
- in the above $p_y(\mathbf{x})$ is the probability the model assigns to $\mathbf{x}$ being of class y
- the reweighted loss is then $\mathcal{L}_w(\mathbf{x}, y) = w_y \mathcal{L}(\mathbf{x}, y)$, where $w_y \propto (\frac{N_y^\lambda}{N})^{-1}$
- we pick each w_y s.t. $\frac{1}{C} \sum_{i=1}^{C} w_i = 1$ to keep the expected loss scale approximately constant
- in practice, this means that the model focuses much more on infrequent classes

3.2.3 Focal Loss

- the focal loss is defined as $\mathcal{L}_{\text{focal}}(\mathbf{x}, y) = -w_y(1 - p_y(\mathbf{x}))^\gamma \log p_y(\mathbf{x})$ where γ is a hyper-parameter
- w_y is a class-specific weight and is either picked $w_y = 1, y = 1, \ldots, C$ (unweighted Focal Loss) or $w_y \propto (\frac{N_y^\lambda}{N})^{-1}$ s.t. $\frac{1}{C}\sum_{i=1}^{C} w_i = 1$ (inverse class frequency weighted Focal Loss)
- the "focal" factor $(1-p_y(\mathbf{x}))^\gamma$ allows the training to focus more on difficult examples
- this is because for examples where $p_y(\mathbf{x}) \approx 1$ (i.e., examples that are already easily classified) the factor will be $(1-p_y(\mathbf{x}))^\gamma \approx 0$ and thus these examples do not contribute much to training
- γ allows to tune the strength of the contribution of the focal factor
- notice that when $\gamma = 0$ the focal loss is equivalent to the (weighted) cross entropy loss

3.2.4 OkO

- Odd-k-out learning, in short OKO, is a method that changes the training framework and can be used on top of any model architecture
- it promises improved calibration for the models it yields, while not compromising on accuracy
- OKO tries to achieve this by training the model on sets of data, rather than individual examples
- in particular, it constructs sets that contain a majority class and the model's task is to identify that majority class
- specifically, given the hyper-parameter "OKO-k" (which will be called k in the subsequent text and should not be confused with the hyper-parameter of KNN), the sets are created in the following way:
  1. a majority class, called "pair class" y, as the set will contain a pair of samples of that class, is selected at random from all available classes
  2. two distinct data points x_1 and x_2 are sampled at random from all available data from class y
  3. data points $x_3, \ldots, x_{k+2}$ are selected at random; each belonging to a unique class $y_3, \ldots, y_{k+2}$ from $\{1, \ldots, C\} \backslash \{y\}$ the set of remaining classes different to y
  4. the resulting tuple (called an "OKO set") of size $k + 2$ is then $\mathcal{S} = ((x_1, y), (x_2, y), (x_3, y_3), \ldots, (x_{k+2}, y_{k+2}))$

- for convenience, given an OKO set $\mathcal{S}$ constructed as given above, we define $\mathcal{S}_x = (x_1, \ldots, x_{k+2})$ and $\mathcal{S}_y = (y, y, y_3, \ldots, y_{k+2})$
- for each OKO epoch, at most N sets are sampled, which means that an equivalent number of gradient updates is performed in each OKO epoch and each epoch in the mitigation strategies previously presented
- the model, given by f_θ, is then trained to identify the majority class y in the OKO set $\mathcal{S}$
- for this, we first define $f_\theta(\mathcal{S}_x) = \sum_{i=1}^{k+2} f_\theta(x_i)$
- this is done by optimizing the "OKO hard loss", given in Eq. 2

$$\ell_{\text{OKO}}(\mathcal{S}_y, f_\theta(\mathcal{S}_x)) = -\mathbf{e}_y^\top \log \text{softmax}(f_\theta(\mathcal{S}_x)) \tag{2}$$

- in Eq. 2 $\mathbf{e}_a \in \mathbb{R}^C$ is an indicator vector at index a
- notably, this is equivalent to unweighted cross entropy training on the logits output by $f_\theta(\mathcal{S}_x)$

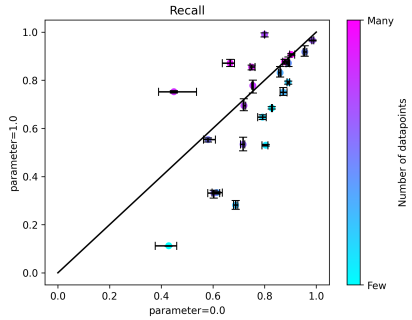
4 Results

- in all of the above settings (i.e., the different class orders, as well as the different mitigation strategies), we train an MLP with a single 100 neuron hidden layer followed by a non-linearity on top of frozen "Virchow_2" features
- we train for 500 epochs (or an equivalent amount gradient updates in the cases of Batch Balancing and OKO), with a fixed learning rate, a fixed weight decay factor, a batch size of 64, and using SGD
- we always pick the OKO-$k = 1$
- these fixed hyper-parameters help comparability across settings

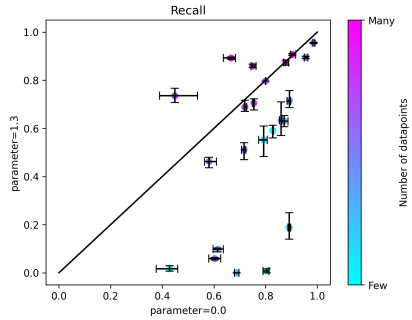
4.1 Original Class Size Order

- Figure 4 shows the mean recall values of all classes at $\lambda = 1.0$ and $\lambda = 1.3$ compared to the balanced setting of $\lambda = 0.0$ over three seeds
- additionally, error bars show the minimum and maximum recall values in each setting over three seeds
- as discussed in Section 3.1.1, the distribution along the x-axis can be seen as a proxy for inherent class difficulty

- in Figure 4a we can see that frequent easy classes sometimes benefit a lot from being overrepresented, while difficult and infrequent classes drop off a lot in the setting $\lambda = 1.0$, which can be seen by looking at the clusters of dots in the upper right quadrant, as well as the dots in the bottom half below $x = 0.7$
- this plot also confirms something intuitive: the more inherently difficult a class is, the more it will struggle under data scarcity, while it potentially profits a lot from overrepresentation
- this is evident from the two dots at approximately $x = 0.4$ in Figure 4a (which both are of a similar inherent class difficulty)
- one of these two classes is a high-frequency class, while the other is a low-frequency class
- both show vastly different recall values in the $\lambda = 1.0$ setting
- as seen in Figure 3c, in the setting $\lambda = 1.3$, many classes have only 1 or 2 samples in the training set, which strongly hinders the learning capabilities of these classes in this setting



(a) $\lambda = 0.0$ vs. $\lambda = 1.0$
Classes of similar inherent difficulty will fall on opposite sides of the main diagonal, when being over- or under-sampled.



(b) $\lambda = 0.0$ vs. $\lambda = 1.3$
Independent of inherent class difficulty, a model will not be able to accurately classify a class if there are not sufficient samples.

Figure 4: Scatter plots comparing the recall values at different λ (Original Class Size Order)
We showcase two settings: One where most infrequent classes can still be classified at reasonable accuracy ($\lambda = 1.0$, Figure 4a) and one where most infrequent classes drop off completely ($\lambda = 1.3$, Figure 4b).

4.2 Easy to Difficult

- Figure 5 shows the same scatter plot as seen in the previous section, but for the setting $\lambda = 0.0$ vs. $\lambda = 1.0$ with the "easy to difficult" class ordering, where easy classes are oversamples and difficult classes are undersampled
- One can visually identify three clusters of dots in this plot
- the first one consists of dots in the center of the bottom half of the plot, corresponding to moderately difficult to highly challenging classes with few samples
- it can be seen that difficult classes suffer a lot from being undersampled (much more than other classes, as can be seen from the clusters discussed below)
- the second cluster contains dots in the center of the plot, aligning closely with the main diagonal
- this cluster consists of dots with a moderate number of samples, and that are moderately difficult to classify
- as the number of samples drops off exponentially with rising class index, one would expect the drop-off in recall along the y-axis to also be steeper
- one hypothesis for why this does not happen is that for these classes a few core examples suffice for the model to learn the class at a mediate accuracy
- as long as these core examples are present in both settings, a similar class accuracy can be reached
- this hypothesis could be investigated by evaluating the recall over the random seeds in the imbalanced sampling step
- the last cluster is in the upper left corner of the plot
- these are inherently easy classes with a high class frequency in the imbalanced setting
- as expected, providing the model with even more samples of classes that are already highly accurately classified in the balanced setting leads the model to perform at least equally well on these classes, without yielding noteworthy improvements

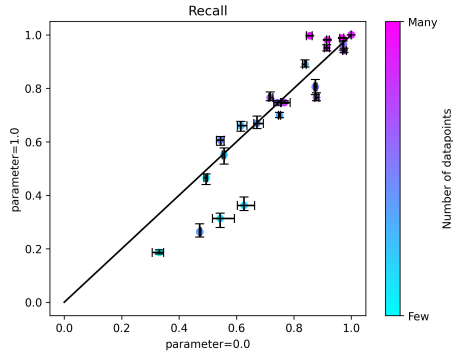


Figure 5: $\lambda = 0.0$ vs. $\lambda = 1.0$ (Easy to Difficult)
The most inherently challenging classes suffer when they are undersampled.

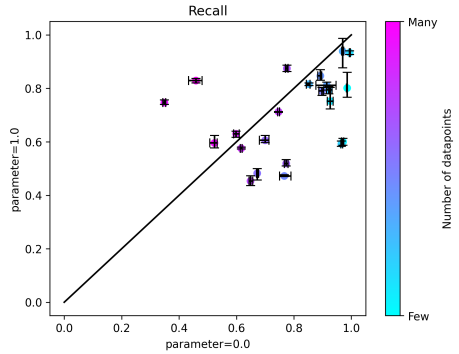


Figure 6: $\lambda = 0.0$ vs. $\lambda = 1.0$ (Difficult to Easy)
Oversampling specifically difficult classes leads to considerable performance gains, without significant costs on easy underrepresented classes.

4.3 Difficult to Easy

- Figure 6 shows the same results in the "difficult to easy" setting
- We see that the most difficult classes seem to have the most to gain by being oversampled
- at the same time, most easy classes do not drop off in performance very significantly by being underrepresented
- this would therefore be the desirable setting in real-world scenarios, and mitigation strategies try to artificially nudge an imbalanced setting to a setting more akin to this (e.g., by oversampling rare classes)

4.4 Performance Changes under Imbalance Mitigation

- Figure 7 compares the results of the different mitigation strategies with the results in the vanilla setting, as well as in the balanced setting ($\lambda = 0$)
- the three methods that actively try to re-balance the distribution of the data (Weighted Cross Entropy, Batch Balancing, Weighted Focal Loss) improve the undersampled classes by noticable margins
- the margins by which they improve the recall is roughy the same on all three methods
- however, they never bring performance back up to the level in the balanced setting
- on some overrepresented classes, they even decrease performance
- this is expected, as fewer samples of these classes are seen during training, compared to the vanilla or balanced setting
- the spread of the recall values in the imbalanced setting under the use of mitigation strategies is indicative of the fact that performance is still largely affected by class imbalance, rather than inherent class difficulty, which is the main driver of performance in the balanced setting, where the variance of recall values is much smaller
- the unweighted focal loss only helps the model to focus on difficult examples during training, and so we cannot expect any significant improvement specifically on underrepresented classes
- through this mitigation strategy, we would rather expect improvements on those classes that were most difficult to classify in the Vanilla setting (imbalanced with parameter $\lambda = 1.0$)
- Figure 7 shows minor improvements on three of the most difficult classes, but overall, no strong effect is present
- another point of note is that OKO does not yield meaningful improvements compared to the vanilla setting
- this is in some way expected, since it promises comparable accuracies and improved calibration, the latter of which was not investigated here

5 Conclusion and Next Steps

5.1 Learnings about Effects of Imbalance

-

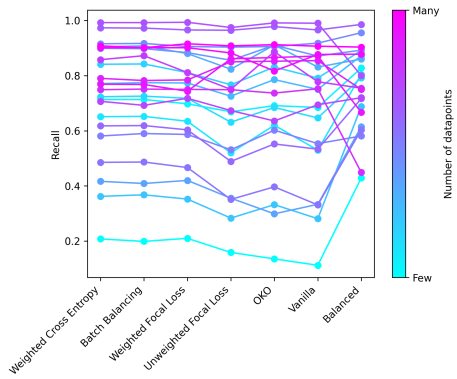


Figure 7: Performance of different Mitigation Strategies in the $\lambda = 1.0$ Setting (Original Class Size Order)
No mitigation strategy recovers the performance on low data classes from the balanced setting. OKO performs surprisingly badly on low data classes.

5.2 Learnings about Mitigation Strategies

- all evaluated mitigation strategies have shortcomings in two potential ways:
  1. most of them do not take inherent class difficulty into account, and they do not have a very good way estimating inherent class difficulty. More specifically, the class distribution is merely rebalanced, and all classes contribute equally to training, but this is completely independent of class difficulty
  2. mitigation strategies can only rebalance by increasing the contributions/weights of individual samples, which can be achieved by sampling a particular sample more often or increasing its contribution to the loss. This does not increase the diversity of samples and thus gains in the generalization of the model are limited.
- OKOs comparatively bad performance could be due to a combination of the following two factors:
  1. while the number of overall gradient steps is the same compared to other methods, the magnitude of each gradient is, on average, multipled by a factor of $k + 2$. This implies a new learning rate should be picked.
     - this is because we perform gradient updates on $f_\theta(\mathcal{S}_x) = \sum_{i=1}^{k+2} f_\theta(x_i)$
  2. the gradients of OKO are obtained from $k + 2$ samples, while they are obtained from a batch (with a batch size B different to $k + 2$) of samples in other mitigation strategies. If $k+2 \ll B$, this might mean

that the gradient updates of OKO are more noisy, and consequently, different hyper-parameters need to be picked.

5.3 Next Steps

- more detailed evaluation using metrics other than just recall
- hyper-parameter tuning (particularly important for OKO)
- investigating calibration

References

[1] Daisuke Komura et al. "Universal encoding of pan-cancer histology by deep texture representations". In: *Cell Reports* 38.9 (2022).